AWS Certified Generative AI Developer – Professional

AIP-C01 · Combined Revision Notes

All 13 lessons in one document

- Bedrock inference, economics, Knowledge Bases, agents, and guardrails
- Prompt management, evaluation, customization, and vector stores
- Integration, security, observability, SageMaker, and the recognition sweep
- Decision rules, troubleshooting patterns, and hands-on checklists

Use the decision rules for revision, then the troubleshooting tables. Work through the checklists to practice each changed path.

Contents

01 — Bedrock Inference APIs	3
02 — Inference Economics	5
03 — Amazon Bedrock Knowledge Bases (managed RAG)	8
04 — Bedrock Agents + AgentCore	11
05 — Amazon Bedrock Guardrails	15
06 — Prompt Management + Prompt Flows	18
07 — Model Evaluation + Customization	21
08 — Vector Stores Outside Knowledge Bases	24
09 — Integration Glue	27
10 — Security Stack	30
11 — Observability + Cost	33
12 — SageMaker AI Family	36
13 — Recognition Sweep	39

01–07: Bedrock hands-on topics. 08–12: supporting platforms. 13: recognition sweep.

01 — Bedrock Inference APIs

Exam focus: how applications actually call foundation models, and which API shape fits a stated requirement.

Control plane vs data plane

API family	Purpose
bedrock	Control plane: models, provisioned throughput, customization jobs
bedrock-runtime	Inference: InvokeModel, Converse, streaming variants
bedrock-agent	Build-time: create agents, knowledge bases, prompts, flows
bedrock-agent-runtime	Runtime: InvokeAgent, Retrieve, RetrieveAndGenerate, ApplyGuardrail lives in bedrock-runtime

Converse vs InvokeModel

	Converse	InvokeModel
Request format	Unified message schema across models	Provider-specific JSON body
Switching models	Change modelId only	Rewrite the request body
Tool use	Native toolConfig	Manual, per provider
Streaming	ConverseStream	InvokeModelWithResponseStream
Exam cue	"Portable", "multi-model", "no code changes when swapping models"	Model-specific parameters Converse doesn't expose

Key Converse fields: messages, system, inferenceConfig (maxTokens, temperature, topP, stopSequences), toolConfig, additionalModelRequestFields (escape hatch for model-specific params), guardrailConfig.

Tool use loop (model never executes anything)

1. Request includes toolConfig: tool name, description, JSON input schema.
2. Model replies with stopReason: "tool_use" and a toolUse block (name + arguments it chose).
3. **Your application** executes the tool (Lambda, API, database).
4. You return the result as a toolResult user message.
5. Model produces the final answer.

Agents (Lesson 04) automate this loop; Converse tool use is the manual version.

Streaming and async

- Real-time token delivery: ConverseStream / InvokeModelWithResponseStream. Pairs with API Gateway WebSockets or server-sent events.

- No native async invoke on Bedrock. For bulk offline work use **batch inference**: JSONL in S3 → CreateModelInvocationJob → results in S3. Discounted per-token pricing (model-dependent), latency in hours.
- Application-level async (user submits, gets notified later): SQS + Lambda pattern you build yourself.

Facts that decide questions

- **Model access**: most serverless models are available on demand; some third-party models require a use-case form / AWS Marketplace subscription before first invocation.
- **Data handling**: Bedrock does not use your prompts/responses to train models. Data stays in-region unless you use a cross-region inference profile.
- **Invocation logging**: opt-in setting that logs full requests/responses to S3 or CloudWatch Logs. This is the audit/compliance answer.
- **Quotas**: on-demand accounts have tokens/requests-per-minute limits. ThrottlingException → exponential backoff (SDK default), quota increase, cross-region profile, or provisioned throughput.
- Temperature/topP are behavior controls, not cost controls.

Decision rules

Scenario wording	Answer pattern
Swap models without code changes	Converse (+ AppConfig/inference profile routing)
Process millions of documents overnight	Batch inference
Typing effect / real-time UX	ConverseStream
Model capacity unavailable in our region	Cross-region inference profile (Lesson 02)
All data must remain in one region	Regional on-demand endpoint; avoid cross-region profiles
Audit every prompt and response	Enable model invocation logging

Hands-on checklist

☐ Call the same prompt via InvokeModel and Converse; compare request bodies.

☐ Run a two-step tool-use conversation by hand.

☐ Stream a response with ConverseStream.

☐ Enable invocation logging and find the record in CloudWatch Logs/S3.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/inference.html

02 — Inference Economics

Exam focus: match the purchasing/optimization model to the traffic pattern in the scenario. The traffic pattern matters more than the model name.

Cost of a request ≈ input tokens × input rate + output tokens × output rate (rates differ; output usually costs more).

Four ways to buy inference

Mode	Billing	Best for	Trap
On-demand	Per token	Variable, spiky, unpredictable traffic	Account/regional quotas; throttling at peak
Provisioned Throughput	Hourly per Model Unit (MU)	Sustained, predictable, high volume	You pay while idle; not automatically cheaper
Batch inference	Discounted per token (model-dependent)	Large async jobs over hours	Not for interactive apps
Prompt caching	Cache-write premium + discounted cache-read rate	Repeated long prompt prefixes	Caches prompt processing, not the final answer

Provisioned Throughput facts

- One MU = a model-specific amount of input tokens/min + output tokens/min. Estimate **both** peaks; an MU is not a number of requests.
- Billed hourly until deleted. Commitments: none, 1 month, 6 months (longer = cheaper hourly rate).
- A customized (fine-tuned/distilled) Bedrock model **requires** Provisioned Throughput to invoke.
- Traffic beyond purchased MUs can still throttle.
- Break-even only at sustained high utilization. Spiky workload → on-demand wins.

Inference profiles (passed anywhere a modelId is accepted)

Cross-region (system-defined)

- AWS-defined set of regions; IDs start with a geography prefix (us., eu., apac.).
- Routes requests to regional capacity → availability and throughput headroom.
- Requests may be processed in another region in that geography → conflicts with strict single-region data-residency requirements.
- Does **not** reserve capacity; it is not Provisioned Throughput.

Application inference profile

- You create it around one regional model or one cross-region profile.

- Purpose: **cost attribution and usage metrics** — tags (team/app/tenant) flow into cost reporting; CloudWatch metrics per application.
- Exam trap: "Track Bedrock spend separately for three business units" → application inference profiles + tags, not three Provisioned Throughputs.
- It adds no capacity by itself.

Prompt caching

- Caches the processed representation of a repeated prompt **prefix**; the model still generates a fresh answer.
- Structure prompts static-first, dynamic-last:

```
Static system instructions / tool definitions / reference doc / few-shot examples
──────────────────────────────── cache checkpoint
Dynamic user input
```

- Explicit caching: model-specific cache checkpoints/controls. Some models also do implicit caching automatically.
- Cache hit needs an exact prefix match; changing an early token invalidates everything after it.
- Reads are cheaper and faster than normal input tokens; writes may cost more than standard input tokens.
- TTL is model-specific (commonly 5 min, sometimes 1 h); hits can refresh it. Hits are best-effort, never guaranteed.
- Monitor cache-read / cache-write token fields in response usage.
- Good candidates: long system prompts, tool definitions, shared documents, few-shot examples. Bad: unique prompts, fast-changing prefixes, infrequent traffic.

Prompt caching vs application-level caching

Technique	Matches	Returns/skips	Main risk
Prompt caching	Exact prompt prefix	Skips reprocessing cached input; model still answers	Miss/expiry
Exact response cache	Hash of normalized request + model + prompt version + params	Previous complete response	Stale answer; tenant leakage if key is wrong
Semantic cache	Embedding similarity of the question	Answer to a similar question	Similar question, different correct answer
Edge/precompute	Known predictable request	Pre-generated content	Staleness

Cache keys must include tenant/user, model, prompt version, and knowledge version where relevant. Never share a semantic cache across tenants for personalized/authorized content.

Model routing and cascading

- **Static routing**: config picks the model per task/tenant (AppConfig works well).
- **Rule-based**: length, language, modality, complexity signals.
- **Classifier routing**: a small model estimates difficulty.

- **Cascade**: cheap model first; low confidence or failed validation → escalate to a stronger model. Savings depend on the escalation rate; a bad cascade invokes two models and costs more.
- Routing also provides fallback during throttling/model unavailability.

Token reduction (usually the cheapest lever)

- Retrieve fewer, sharper chunks; metadata-filter before retrieval; cap topK.
- Summarize old conversation turns instead of resending full history.
- Set maxTokens; request concise/structured output.
- Remove duplicated instructions. (Temperature is not a cost lever.)

Decision rules

Scenario wording	Likely answer
Sustained predictable traffic, 24/7	Provisioned Throughput
Bursty and unpredictable	On-demand
Separate costs by department/application	Application inference profile + tags
Capacity unavailable in current region	Cross-region inference profile
Same long system prompt/document every request	Prompt caching
Same question phrased differently, repeatedly	Semantic caching (with threshold + TTL + tenant isolation)
Millions of documents, results by tomorrow	Batch inference
Mostly simple queries, a few complex	Routing or cascading to a larger model
Retrieved context too large	Reduce topK, filter, prune context

Hands-on checklist

☐ Create an application inference profile with tags; invoke through it; inspect usage.

☐ Add a cache checkpoint after a long static prefix; compare usage fields and latency across repeats.

☐ Estimate Provisioned Throughput from measured peak input/output token rates.

Sources: https://docs.aws.amazon.com/bedrock/latest/userguide/prov-throughput.html · https://docs.aws.amazon.com/bedrock/latest/userguide/inference-profiles.html · https://docs.aws.amazon.com/bedrock/latest/userguide/prompt-caching.html

03 — Amazon Bedrock Knowledge Bases (managed RAG)

Exam focus: the ingestion → embedding → retrieval pipeline, and the controls that fix retrieval quality.

```
Data source → parse → chunk → embed → vector index
User query → embed query → filter + similarity search → optional rerank → prompt + model → answer + citations
```

Creation choices (expensive to reverse)

Data sources

S3 (most common), web crawler, Confluence/SharePoint/OneDrive/Google Drive connectors, custom/direct ingestion APIs.

Changing source content does nothing until you **sync** (StartIngestionJob). Sync is incremental: added/modified/deleted documents are processed; deletions leave the index.

Parsing

- Default parser: plain text extraction.
- Advanced parsing (Bedrock Data Automation or an FM-based parser): complex PDFs, tables, charts, scanned/visual content.
- Custom transformation Lambda: your own cleaning, redaction, metadata enrichment, or splitting.
- Exam cue: "answers must use the tables/diagrams in our PDFs" → advanced/multimodal parsing, not default parsing alone.

Embedding model

- Same model embeds chunks and queries (same vector space — non-negotiable).
- Selection factors: language/domain fit, dimensions, multimodal support, cost.
- Treat as a creation-time decision: changing it generally means a new index/KB and full re-ingestion.

Vector store options

Store	Choose when
OpenSearch Serverless (KB can auto-create)	Fastest managed setup; semantic + hybrid search
OpenSearch Service managed cluster	Cluster-level control (Lesson 08)
Aurora PostgreSQL + pgvector	Already on Postgres; SQL joins to relational data
Neptune Analytics	Relationship/graph questions (GraphRAG)
S3 Vectors / Pinecone / Redis / MongoDB Atlas	Cost or existing-platform reasons

Newer consoles also offer a fully **Managed Knowledge Base** (AWS runs ingestion/index/storage/retrieval). The mechanics in this lesson apply either way.

Chunking — the highest-impact quality lever

Strategy	How	Best for
Default	~300-token chunks, honors sentence boundaries	Quick start, general docs
Fixed-size	Max tokens + overlap % you set	Uniform documents
No chunking	Whole document = one chunk	You pre-split documents yourself (often via Lambda)
Hierarchical	Small child chunks match precisely, then are replaced by larger parent chunks for context	Long structured docs: manuals, contracts, policies
Semantic	Splits where sentence meaning shifts; uses an FM (extra cost)	Mixed-topic documents

Tradeoffs: too small → precise but context-poor; too large → noisy and token-expensive; overlap preserves cross-boundary context but duplicates content.

Query-time configuration

Control	Rule
numberOfResults (default 5)	More = better recall, more noise/cost/context pressure
Semantic search	Paraphrased, conceptual questions
Hybrid search (keyword + vector; OpenSearch Serverless, Aurora/RDS, MongoDB)	Exact terms matter: SKUs, error codes, names
Metadata filtering	Restrict corpus before ranking: tenant, department, date, type
Reranking	Retrieve wide, reranker re-scores, keep the best few; adds a model call, can reduce prompt cost
Query transformation	Complex/multi-hop questions: decompose (Lambda/Step Functions/Bedrock), retrieve per sub-question, combine

Metadata filtering is **not** authorization by itself: the application must inject tenant/user filters server-side; never trust filter values from the end user.

Retrieve vs RetrieveAndGenerate

API	Returns	Use when
Retrieve	Chunks, scores, source locations	You control the prompt, model call, guardrails, routing, multi-KB merging
RetrieveAndGenerate	Answer **with citations**; supports sessionId for multi-turn	Fastest working RAG app

API	Returns	Use when
RetrieveAndGenerateStream	Streaming version	Real-time UI

Citations improve verifiability; they do not prove faithfulness (grounding checks, Lesson 05/07).

Structured-data KBs: for Redshift/Glue-backed sources, Bedrock converts natural language to SQL and queries tables directly — no embeddings. Cue: "live tabular sales data" → text-to-SQL beats embedding rows.

Troubleshooting patterns

Symptom	Likely cause
Retrieved chunks irrelevant	Chunking, embedding fit, need hybrid search or reranker
Right chunk retrieved, answer wrong	Prompt/generation problem; conflicting/insufficient context
Answers cite outdated policy	Sync never ran / not automated (S3 event → Lambda/EventBridge → StartIngestionJob)
Documents missing	Ingestion or parsing failure; IAM/KMS permissions
Users see other departments' docs	Missing server-side metadata filtering / access design
Table/diagram questions fail	Text-only parsing; need advanced parsing or multimodal embeddings

Decision rules

- Managed RAG with citations, minimal code → Knowledge Bases + RetrieveAndGenerate.
- Custom prompt/model orchestration over chunks → Retrieve + own Converse call.
- Precise match **and** full context in long manuals → hierarchical chunking.
- Exact product codes in queries → hybrid search.
- Restrict to the user's department → server-injected metadata filtering.
- Hourly document changes → automated incremental sync.
- Live relational/tabular data → structured KB (NL→SQL).

Hands-on checklist

☐ Ingest the same PDFs with fixed-size vs semantic/hierarchical chunking; compare retrieved chunks.

☐ Add .metadata.json sidecar files; filter a query by an attribute.

☐ Call Retrieve (inspect scores), then RetrieveAndGenerate (inspect citations).

☐ Modify/delete a source document, sync, verify index behavior.

Sources: https://docs.aws.amazon.com/bedrock/latest/userguide/knowledge-base.html · https://docs.aws.amazon.com/bedrock/latest/userguide/kb-chunking.md · https://docs.aws.amazon.com/bedrock/latest/userguide/kb-test-config.md

04 — Bedrock Agents + AgentCore

Exam focus: when the model should decide the steps, and how to run agents safely in production.

Do you even need an agent?

Situation	Tool
Answer questions from documents	Knowledge Base RetrieveAndGenerate (Lesson 03)
Steps known and fixed in advance	Step Functions / Prompt Flows — deterministic, cheaper
Open-ended task; model must choose tools and order	Agent

Agents trade determinism and cost for flexibility: every reasoning step is another FM call.

Bedrock Agents (managed orchestration)

An agent is a configured resource, not code:

```
Agent = foundation model + instructions + action groups (tools)
      + knowledge bases (optional) + guardrail (optional) + memory config (optional)
```

Orchestration loop (ReAct: reason → act → observe)

```
User input → pre-processing (categorize/validate)
  → agent reasons → query KB | call action group | ask user (clarification)
  → observes result → reasons again … → post-processing → final answer
```

Stops when done, at step limits, or on failure — hence exam emphasis on stopping conditions, timeouts, circuit breakers.

Action groups

- Defined by a **schema** (OpenAPI schema for API operations, or function schema: names + parameters only) plus an **executor**.
- **Lambda executor**: Bedrock calls your Lambda with the action name and model-extracted parameters. The classic, most-tested pattern.
- **Return of control**: Bedrock returns the requested call to *your application* in the InvokeAgent response; you execute it (systems only your app tier can reach, extra validation) and pass the result back.
- Built-in groups: **User input** (agent asks clarifying questions), **Code interpretation** (agent writes/runs code in a Bedrock sandbox).
- Quality lever: **schema descriptions are prompts.** Vague parameter descriptions → wrong/hallucinated arguments. First fix for "agent calls the tool wrong", together with tracing.

Memory — three different things

Type	Mechanism	Scope
In-session context	Same sessionId across InvokeAgent calls	One conversation
Session attributes	SessionState passed in/out per request	App-injected data (user tier, cart)
Agent memory	Bedrock stores **session summaries** per user (memoryId), configurable retention	Cross-session recall

DIY alternative: DynamoDB conversation history (Lesson 09). AgentCore Memory: framework-agnostic version (below).

Deployment lifecycle

```
DRAFT (test alias TSTALIASID) → Prepare → Version (immutable snapshot) → Alias → InvokeAgent
```

- Alias can weight-split traffic between two versions (canary).
- **Inline agents** (InvokeInlineAgent): model/tools/instructions defined in the request; nothing persisted. Cue: "ephemeral per-request/per-tenant agents".
- **Tracing**: enableTrace returns rationale, action inputs, KB queries, observations per step. Primary debugging tool for agent misbehavior.

Multi-agent collaboration

Supervisor agent delegates to specialized collaborator agents (own tools/KBs/permissions):

- **Supervisor mode**: supervisor orchestrates every step — capable, expensive.
- **Supervisor-router mode**: classifier routes the request to one collaborator — faster, less flexible.

Use when domains are genuinely separate (billing vs tech support), not to organize what one agent with several action groups could do.

AgentCore (bring your own framework)

Bedrock Agents = AWS orchestrates, configured, Bedrock models. **AgentCore = production primitives for agents you build** — any framework (Strands, LangGraph, CrewAI), any model. Modular, usable independently:

Component	Job
Runtime	Serverless agent hosting; isolated session per user (own microVM), long-running sessions; HTTP/MCP/A2A protocols
Memory	Short-term session events + long-term memory (summaries, semantic facts, user preferences) for any framework
Gateway	Wraps existing Lambda functions / OpenAPI APIs / API Gateway endpoints as **MCP tools** — no rewrite
Identity	Inbound auth (who may invoke) + outbound credentials (agent calls SaaS **on behalf of the signed-in user**, OAuth)

Component	Job
Browser Tool	Managed cloud browser for web-interaction tasks
Code Interpreter	Sandboxed execution of agent-generated code
Policy	Deterministic (Cedar) rules on Gateway tool calls — control outside the model's judgment
Observability / Evaluations	OpenTelemetry traces to CloudWatch; online agent quality evaluation

Names to recognize

- **MCP (Model Context Protocol)**: open standard for agent↔tool discovery and calls. Exam rule: stateless MCP servers on **Lambda** for lightweight tools; **ECS** for complex/long-running ones.
- **A2A**: agent-to-agent protocol.
- **Strands Agents**: AWS open-source, model-driven agent SDK.
- **Agent Squad**: AWS multi-agent orchestration framework.

Decision rules

Scenario	Answer
Create records, check order status from user intent	Bedrock Agent + Lambda action group
Existing REST APIs must become tools for MCP agents	AgentCore Gateway
Agent built in LangGraph needs secure scalable hosting	AgentCore Runtime
Agent acts on the user's own Google Drive/permissions	AgentCore Identity (outbound OAuth)
Agent must browse a website / run generated Python	Browser Tool / Code Interpreter
Fixed, auditable steps	Step Functions, not an agent
Remember returning users across sessions	Agent memory / AgentCore Memory (DIY: DynamoDB)
Agent loops or calls wrong tool	Enable trace; fix schema descriptions; iteration limits; guardrails
Sensitive action needs approval first	User confirmation / return-of-control, or Step Functions human-approval

Hands-on checklist

- ☐ Bedrock Agent with one Lambda action group; invoke with enableTrace; read the reasoning steps.
- ☐ Attach a Knowledge Base; observe when it chooses KB vs action.
- ☐ Publish version 1 behind an alias; test changes on TSTALIASID.
- ☐ Expose one Lambda/API through AgentCore Gateway as an MCP tool.

Sources: https://docs.aws.amazon.com/bedrock/latest/userguide/agents-how.html · https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/what-is-bedrock-agentcore.html

05 — Amazon Bedrock Guardrails

Exam focus: one policy layer filtering what goes into and out of any model. Each policy type solves a different problem — the exam tests telling them apart.

The six policies

Policy	Mechanism	Use for
Content filters	Predefined categories (Hate, Insults, Sexual, Violence, Misconduct, **Prompt Attack**), strength none→high, set separately for input/output	Harmful content; prompt injection/jailbreak attempts
Denied topics	Topics defined in natural language + example phrases; detected **semantically**	"Never discuss competitors / no personalized investment advice" — even paraphrased
Word filters	Exact whole-word/phrase match (case-insensitive); managed profanity list available	Specific strings: profanity, competitor names
Sensitive information filters	Probabilistic PII detection + custom regex; action = **block** or **mask/anonymize**	Redact SSN, email, account numbers in inputs, outputs, transcripts
Contextual grounding checks	Scores response vs retrieved source + query: grounding (supported by source?) and relevance (answers the question?); threshold-based	Hallucinated RAG answers
Automated Reasoning checks	Validates response claims against a formal logic policy built from your rule documents; returns findings/corrections	Deterministic correctness: eligibility, policy rules, calculations

Distinctions that decide questions

- **Denied topics ≠ word filters**: semantic topic vs exact string. A competitor can be discussed without being named.
- **Grounding ≠ fact-checking**: grounding verifies against *your provided source*, not the world.
- **Masking ≠ blocking**: masking keeps the conversation usable ("My SSN is {SSN}"); blocking rejects. Inputs can be blocked or anonymized pre-model; outputs are typically masked.

Three ways to invoke

1. **Attached to inference** — guardrailIdentifier + version in Converse/InvokeModel; also attachable to Agents and Knowledge Base queries. Prompt and completion evaluated automatically.
2. **ApplyGuardrail API** — evaluates arbitrary text with **no model involved**. Critical uses:
   - Screen user input *before* it reaches a model, agent, or retrieval step.
   - Validate output from models Bedrock didn't run (SageMaker endpoint, self-hosted, third-party).

3. **Selective evaluation (tagging)** — mark only the user-supplied segment of a prompt for evaluation so system instructions, retrieved docs, and history aren't treated as "user input".

Lifecycle

- Edit the **DRAFT**; production pins a **numbered version**. Updating a draft never silently changes production.
- Configure custom blocked-input / blocked-output messages.
- Trace output shows which policy intervened (action: INTERVENED) — debugging and audit trail.

Defense in depth (Domain 3 pattern)

Guardrails are one layer, not the whole design:

```
API Gateway / WAF (rate limits, payload size)
  → optional pre-filter (Comprehend PII, Lambda validation/sanitization)
  → Guardrail on input (prompt attack, denied topics, PII)
  → Knowledge Base / Agent / Model
  → Guardrail on output (content, PII masking, grounding check)
  → Lambda post-validation (JSON schema, business rules)
  → response
```

Guardrails do not replace IAM, network isolation, or schema validation; no filter catches every jailbreak — which is why layered designs are tested.

Decision rules

Scenario	Answer
Moderate text before any model call / from an external model	ApplyGuardrail
Topic must never be discussed, however phrased	Denied topics
Block these exact words	Word filters
Stored transcripts must not contain PII	Sensitive info filter with masking
RAG answers invent unsupported details	Contextual grounding checks (+ retrieval fixes, Lesson 03)
Verify decisions against formal policy rules	Automated Reasoning checks
Attackers override the system prompt	Content-filter prompt-attack category, inside the layered design
Different behavior test vs prod	DRAFT in test, pinned version in prod

Hands-on checklist

- ☐ Create a guardrail (denied topic + PII masking + content filters); exercise the test window.
- ☐ Call ApplyGuardrail on raw text — confirm block/redact with no model involved.
- ☐ Attach to a Converse call; trigger an intervention; read the trace.
- ☐ Run an ungrounded answer through a grounding check at different thresholds.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails.md

06 — Prompt Management + Prompt Flows

Exam focus: prompts as versioned, testable configuration — not strings buried in code — and chaining them into workflows.

Part A — Prompt Management

A managed prompt bundles: **template text + variables + model + inference settings**. Changing any of them creates a new **variant** — the unit you test and compare.

- **Variables**: {{placeholder}} values filled at test/runtime. One template, many use cases.
- **Variants**: alternative wording, model, or temperature/topP for the same prompt. Test window runs variants side-by-side on identical inputs.
- **Versions**: immutable snapshots. Applications reference a specific version; draft edits never affect production. Rollback = point back to the previous version.
- **Prompt optimization**: Bedrock can auto-rewrite a prompt for a chosen model; keep the result as a variant.
- **Prompt caching** can be enabled on the prompt (Lesson 02) — natural home for long, stable system prompts.
- Usable directly as a Converse target, or as a prompt node inside a Flow.

Governance angle: centrally owned templates, controlled promotion draft→version, usage tracked via CloudTrail/CloudWatch. Anti-pattern tested against: prompts hardcoded in application code.

Prompt-craft techniques to recognize

- Role/system definition + explicit output-format spec ("return only this JSON schema")
- Structured, delimited input sections separating instructions from data
- Few-shot examples for format/behavior consistency
- Chain-of-thought instructions for multi-step reasoning
- Iterative refinement driven by evaluation feedback (Lesson 07)

Part B — Prompt Flows

A directed graph of nodes; node outputs wire into inputs via expressions (JSONPath-style) with type validation.

Node catalog

Category	Nodes
Logic	Input, Output, **Condition** (branching; first match wins), **Iterator/Collector** (fan array out and back), **DoWhile** (loop with maxIterations)
Work	**Prompt** (managed or inline; guardrail attachable), **Knowledge base** (retrieve-only, or RetrieveAndGenerate with guardrails), **Agent** (can pause mid-flow for user input — multi-turn), **Lambda**, **Inline code** (small Python transforms), **S3 retrieval/storage**

Lifecycle (same alias/version discipline as agents and guardrails)

```
Working draft → Prepare → test → publish Version (immutable) → Alias (can weight-split versions) → InvokeFlow
```

Long flows can run **asynchronously** with execution tracking; synchronous invocation streams node events.

Flows vs Step Functions vs Agents

	Prompt Flows	Step Functions	Agents
Who decides steps	You, visually, in advance	You, in advance	The model, at runtime
Strength	Prompt/KB/agent chaining, almost no code	Enterprise orchestration: retries, error handling, human approval, long waits, broad service integrations	Open-ended tasks
Weakness	Bedrock-centric; limited general orchestration	No native prompt/KB nodes (wire Lambda/Converse yourself)	Cost, latency, nondeterminism
Exam cue	"Chain prompts with branching, minimal code"	"Auditable workflow with approval step and retries"	"Model decides which actions to take"

Decision rules

Scenario	Answer
Same prompt across apps; changes reviewed and reversible	Prompt Management + versions
Compare two prompt wordings/models before choosing	Variants in the test window
Classify request, route to a specialized prompt	Flow + Condition node
Summarize each document in a list, then combine	Iterator → Prompt → Collector → Prompt
Workflow needs manager approval + retry logic	Step Functions, not Flows
Steps unknowable in advance	Agent (Lesson 04), not a Flow

Hands-on checklist

- ☐ Prompt with two variables; second variant on another model/temperature; compare; publish a version.
- ☐ Flow: Input → Knowledge base node → Prompt node (managed prompt) → Output; publish version + alias; invoke.
- ☐ Add a Condition node routing on a classification result.

Sources: https://docs.aws.amazon.com/bedrock/latest/userguide/prompt-management.md · https://docs.aws.amazon.com/bedrock/latest/userguide/flows.md

07 — Model Evaluation + Customization

Exam focus: prove a model/prompt works, then decide whether to change the model itself — and how.

Part A — Evaluation

Three evaluation methods

Method	Scoring	Cost/speed	Use when
Programmatic (automatic)	Deterministic metrics vs expected outputs: accuracy, ROUGE/BERTScore/F1-style similarity	Cheapest, fastest	Objectively checkable answers; regression testing
LLM-as-a-judge	A second model scores + explains: correctness, completeness, **faithfulness**, helpfulness, coherence, instruction-following; responsible-AI metrics (harmfulness, stereotyping, refusal)	Moderate; scales to thousands	Open-ended generation with no exact-match answer
Human evaluation	Your employees/SMEs rate outputs via managed work teams	Slowest, most expensive	Subjective quality, brand voice, safety; calibrating the judge

Production pattern: **judge at scale, humans on a sample** to validate the judge.

Mechanics

- Input: custom prompt dataset (JSONL in S3) for your use case, or built-in benchmark datasets.
- Evaluable: foundation models, fine-tuned/imported models, provisioned-throughput models, inference profiles, prompt routers, **knowledge bases**.
- Output: report card (console + S3) per metric → usable as deployment quality gates ("faithfulness must not regress below X").

RAG evaluation — two separate failure domains

Layer	Question	If it fails, fix
Retrieval	Did we fetch relevant context? (context relevance/coverage vs ground truth)	Chunking, embeddings, hybrid search, filters, reranking (Lesson 03)
Generation	Is the answer correct, complete, **faithful to what was retrieved**?	Prompt, model choice, grounding checks (Lessons 05–06)

Needs ground truth: queries + expected source texts + expected responses. This split drives Domain 5 troubleshooting: never "fix" generation when retrieval is the broken half.

Adjacent patterns

- **A/B and canary**: alias weight-splitting (agents, flows), app-level routing, or offline eval on the same dataset. Compare quality **and** cost/latency (token efficiency, latency-to-quality).
- **Agent evaluation**: task-completion rate, tool-call effectiveness, reasoning quality from traces (Lesson 04; AgentCore Evaluations).

Part B — Customization

Decision ladder — try in order

```
1. Prompt engineering     → fixable with instructions/examples? stop
2. RAG                    → missing or changing facts? stop
3. Fine-tuning            → consistent style/format/task behavior?
4. Continued pre-training → must absorb domain language/corpus?
5. Distillation           → big model right but too slow/expensive at volume?
6. SageMaker self-managed → full control (LoRA/adapters, custom models)?
```

Methods

Method	Training data	Changes	Exam cue
Supervised fine-tuning (SFT)	Labeled prompt→response pairs (JSONL in S3)	Behavior: format, tone, task pattern	"Always output our exact ticket-classification format"
Reinforcement fine-tuning	Prompts + a **reward function** (Lambda) scoring responses	Alignment with your quality criteria, learned iteratively	"Optimize for our business-rule scoring without labeled answers"
Continued pre-training	Large unlabeled domain corpus	Domain vocabulary/knowledge	Deep domain adaptation (model availability varies — verify)
Distillation	Your prompts; **teacher** generates responses, **student** fine-tuned on them (Bedrock automates)	Small model inherits big-model quality on your task distribution	"Frontier quality at small-model cost/latency"
LoRA / adapters (SageMaker)	Labeled data; trains small adapter weights on frozen base	Cheap, swappable adaptation	Self-managed open models; Model Registry versioning/rollback (Lesson 12)

Operational facts

- Fine-tuning cost = training tokens × epochs, plus per-model monthly storage.
- Customized Bedrock models invoke **only through Provisioned Throughput** (Lesson 02) — budget for it.
- Controllable hyperparameters: epochs, learning rate, batch size; training/validation split from your dataset.
- Lifecycle: evaluate customized vs base model before deploying; keep a rollback path to the previous version.

Amazon's own models

- **Titan Embeddings** (text + multimodal): the KB embedding workhorse (Lesson 03). **Titan Image Generator**: image generation/editing.
- **Nova family**: current generation — text/multimodal tiers (Micro/Lite/Pro map to the Lesson 02 cost/capability ladder), plus image/video/speech variants.

Decision rules

Scenario	Answer
Data changes daily; answers cite current documents	RAG, never fine-tuning
Output format inconsistent despite clear instructions	SFT (after prompt engineering fails)
Frontier model works; unit economics fail at scale	Distillation
Nightly scoring of thousands of responses	LLM-as-a-judge
Subjective brand/safety judgments	Human evaluation
RAG quality dropped	Evaluate retrieval and generation **separately** first
Choosing between two models for our workload	Eval job on **your** dataset + cost/latency analysis, not public benchmarks

Hands-on checklist

☐ ~50-prompt JSONL dataset from your Lesson 03 app; automatic eval comparing two models.

☐ LLM-as-judge job; read explanations, not just scores.

☐ RAG evaluation; classify your errors as retrieval vs generation.

☐ (Read-only) Walk the fine-tuning flow: dataset format, hyperparameters, Provisioned Throughput requirement.

Sources: https://docs.aws.amazon.com/bedrock/latest/userguide/evaluation.md · https://docs.aws.amazon.com/bedrock/latest/userguide/custom-models.md

08 — Vector Stores Outside Knowledge Bases

Exam focus: what the Knowledge Base hides from you, and when to build the pipeline yourself.

Why build custom at all?

A KB already uses these stores. Go custom only for control the KB doesn't expose: custom chunking/preprocessing, streaming/near-real-time ingestion, your own hybrid scoring or multi-index/tenant design, or an existing OpenSearch/Postgres platform. Otherwise the exam default is "use a Knowledge Base".

ANN in one paragraph

Exact nearest-neighbor compares the query vector to **every** stored vector — too slow at scale. **Approximate** nearest neighbor (ANN) trades a little recall for large speed gains:

- **HNSW**: graph-based; high recall, fast queries; memory-hungry, slower to build.
- **IVFFlat** (pgvector): clustered; needs existing data to train clusters; cheaper memory; recall depends on probes searched.

Distance metric must match the embedding model's design (Titan/Cohere → cosine is the norm). Mismatched metric = silently degraded retrieval.

OpenSearch — the search specialist

	Managed cluster (OpenSearch Service)	OpenSearch Serverless
You manage	Instances, shards, replicas, index design	Almost nothing; capacity scales via OCUs
Choose when	Tuning control, predictable large scale, cost tuning	Variable load, minimal ops, KB quick-start

Key concepts:

- knn_vector field with fixed dimension matching your embedding model; engine (e.g., FAISS/HNSW).
- **BM25 keyword search is native** → OpenSearch is the hybrid-search answer. Search pipelines merge keyword + vector scores (normalization or reciprocal rank fusion).
- **Neural search**: query text is embedded for you (in-cluster model or Bedrock connector) instead of by your app.
- Scaling levers: shard size/count, replicas (read throughput). k-NN graphs live in memory → watch KNNGraphMemoryUsage vs node RAM in CloudWatch.
- Patterns: separate indices per domain/tenant; metadata as filterable fields (Lesson 03 filtering).

Aurora PostgreSQL + pgvector — the relational option

- vector column + HNSW or IVFFlat index; operators <=> cosine, <-> L2, <#> inner product.
- Killer feature is **SQL**: similarity + WHERE filters + joins to live business tables + tsvector full-text (hybrid) in one query. Metadata filtering is just SQL.
- Aurora scaling: storage auto-scale, read replicas, Serverless v2.

- As a KB store it needs specific wiring: Data API enabled, credentials in Secrets Manager, table with chunk/embedding/metadata columns, HNSW cosine index. Custom RAG can use any schema.
- Choose when data already lives in Postgres and retrieval must join transactional data. Pure vector scale/search sophistication → OpenSearch.

Supporting cast

- **Neptune Analytics**: GraphRAG — retrieval follows entity *relationships* ("which suppliers does this component failure affect?").
- **S3 Vectors**: low-cost S3-native vectors for large, latency-tolerant corpora.
- **DynamoDB**: **not** a vector store. Roles: metadata, conversation history, session state beside a real vector index.

The custom pipeline

```
Ingest:  S3 event → EventBridge/Step Functions → parse + chunk (Lambda)
         → Bedrock embeddings (batch where possible) → bulk index
Query:   app → embed query → hybrid search + metadata filters
         → optional rerank → Converse with retrieved context
Change:  source change → re-embed changed docs only
         embedding-model change → NEW index, dual-run, cut over (blue/green)
```

Embeddings from different models live in different vector spaces. Never mix models in one index or query a Titan index with Cohere vectors.

Decision rules

Scenario	Answer
Full control of indexing, shards, hybrid scoring	OpenSearch managed cluster
Vector search, zero cluster ops	OpenSearch Serverless
Join semantic search to live customer/order tables	Aurora pgvector
Questions about relationships between entities	Neptune GraphRAG
Millions of rarely queried vectors, cost first	S3 Vectors
Standard RAG, no search expertise on team	Knowledge Base (Lesson 03), not custom

Troubleshooting

Symptom	Likely cause
Quality collapsed after a "small change"	Embedding model/dimension changed without re-indexing
Slow queries at scale	Shard/replica sizing; k-NN graph over node memory; IVFFlat probes too low

Symptom	Likely cause
Exact-match terms not found	Semantic-only search → switch to hybrid
Fresh documents not searchable	Ingestion lag/failure; refresh/sync not wired
Filtered queries return too little	Over-restrictive metadata filters before ANN search

Hands-on checklist

- ☐ OpenSearch index with knn_vector; kNN query, then a hybrid query.
- ☐ Aurora: CREATE EXTENSION vector, vector column, HNSW index, cosine query with a SQL WHERE filter.
- ☐ Query an index with the wrong embedding model once — observe garbage results; remember the vector-space rule.

Sources: https://docs.aws.amazon.com/opensearch-service/latest/developerguide/knn.html · https://docs.aws.amazon.com/AmazonRDS/latest/AuroraUserGuide/AuroraPostgreSQL.VectorDB.html

09 — Integration Glue

Exam focus: the non-AI services that carry GenAI workloads. Most scenario questions are won or lost here, not on model choice.

Lambda — the default compute glue

GenAI roles: agent action groups (Lesson 04), KB custom transformation/chunking, prompt pre/post-processing, query decomposition, stateless MCP servers, API backends, guardrail-adjacent validation.

Know cold:

- Max execution **15 minutes**; long FM/agent calls must fit or move to Step Functions/async patterns.
- Concurrency: account/regional limits, reserved concurrency (protect a critical function), provisioned concurrency (kill cold starts).
- Cold starts matter for chat latency; memory size also buys CPU.
- Synchronous (API request/response), asynchronous (SQS/EventBridge invoke, retries + DLQ), and streaming-response invocation modes.

API Gateway — the front door

Type	Use
REST API	Full features: request validation, usage plans, API keys, caching, WAF integration
HTTP API	Cheaper/simpler JWT-authorized APIs
WebSocket API	Server-driven streaming chat UX (with Bedrock streaming behind Lambda)

GenAI-specific traps:

- Integration timeouts make long-running synchronous FM calls risky → stream, go async (accept → poll/notify), or orchestrate with Step Functions.
- Request validation rejects malformed input before it costs tokens.
- Throttling/usage plans protect the token budget behind the API; API keys are **not** authentication (pair with Lambda authorizer/Cognito/IAM — Lesson 10).

Step Functions — deterministic orchestration

- **Standard** workflows: long-running (up to 1 year), exactly-once, human approval via callback task tokens. **Express**: high-volume, short (≤5 min), cheaper.
- Built-in retries/backoff, catch branches, parallel branches, Map state (fan-out over arrays), Wait states.
- GenAI patterns named in the exam guide:
  - **ReAct / chain-of-thought loops** with explicit stopping conditions (the controllable alternative to autonomous agents).
  - **Circuit breaker** + graceful degradation around FM calls.

  - Query decomposition/transformation for RAG (Lesson 03).
  - Human review/approval steps (with Amazon A2I, Lesson 12).
  - Document-processing pipelines (ingest → extract → validate → index).
- Rule: if steps are known, auditable, and need retries/approvals → Step Functions. Model decides steps → Agent (Lesson 04). Prompt-centric chaining with minimal code → Prompt Flows (Lesson 06).

EventBridge — event plumbing

- Triggers: S3 object events → start KB sync or a processing workflow; scheduled rules for periodic sync/evaluation jobs; SaaS/application events routed to targets.
- Event buses + rules decouple producers from GenAI consumers; Schema Registry and Pipes (point-to-point with filtering/transformation, e.g., SQS→Lambda) are recognition-level.

SQS / SNS — decouple and fan out

- **SQS**: buffer bursty inference requests so Lambda workers process at a controlled rate (protects Bedrock quotas); DLQ for poison requests; FIFO for ordered, deduplicated processing (rarely needed for chat; relevant for sequential document/event processing). Watch visibility timeout vs worst-case FM call duration.
- **SNS**: fan-out notifications — job-complete alerts, multi-consumer events. Not a queue; pair SNS→SQS per consumer for durability.

DynamoDB — state and metadata, not vectors

GenAI roles: conversation history, session state, agent/app metadata, semantic-cache metadata, prompt/feature flags adjunct.

- Design: partition key per user/session; sort key by timestamp; **TTL** to expire sessions automatically.
- Streams → trigger downstream processing (e.g., summarize completed conversations).
- Limits that bite: 400 KB item size → long documents/full histories belong in S3 with pointers in DynamoDB.
- On-demand capacity for spiky chat workloads.

Reference patterns

Interactive chat: CloudFront → API Gateway (WebSocket or REST) → Lambda → Bedrock Converse/ConverseStream (or agent) → DynamoDB session history; guardrail attached at invoke.

Async document pipeline: S3 upload → EventBridge → Step Functions (parse → chunk → embed → index / start KB sync) with SQS buffering and DLQ; SNS notification on completion/failure.

Batch inference front-end: API accepts job → S3 staging → batch inference job (Lesson 01) → EventBridge/SNS completion → results to S3.

Decision rules

Scenario	Answer
Fixed multi-step process with retries and approval	Step Functions (Standard)

Scenario	Answer
Sudden request spikes exhaust Bedrock quota	SQS buffer + controlled Lambda concurrency
Users must be notified when long jobs finish	SNS (or EventBridge targets)
New document in S3 must become searchable	EventBridge → KB sync / ingestion workflow
Chat needs per-user conversation state with expiry	DynamoDB + TTL
Lightweight stateless tool for agents	Lambda MCP server (complex/long-running → ECS, Lesson 04)

Hands-on checklist

- ☐ WebSocket or streaming REST chat endpoint → Lambda → ConverseStream.
- ☐ SQS queue fronting a Bedrock-calling Lambda; observe throttling protection and DLQ behavior.
- ☐ Step Functions workflow: prompt → condition → second prompt, with a retry policy.
- ☐ DynamoDB session table with TTL storing conversation turns.

Sources: https://docs.aws.amazon.com/step-functions/latest/dg/welcome.html · https://docs.aws.amazon.com/apigateway/latest/developerguide/welcome.html · https://docs.aws.amazon.com/lambda/latest/dg/configuration-concurrency.html

10 — Security Stack

Exam focus (Domain 3, 20%): who can call what, where data flows, and how sensitive data is found and protected. Guardrails (Lesson 05) are one layer in this stack, not the stack.

IAM — least privilege for GenAI

- Scope inference permissions to specific resources: bedrock:InvokeModel / bedrock:Converse on named model or **inference profile ARNs**, not bedrock:* on *.
- Separate permissions per operation: invoking models ≠ managing knowledge bases ≠ bedrock:ApplyGuardrail ≠ starting ingestion jobs.
- Every Bedrock resource runs under an **execution/service role**: KB role (read S3, write vector store, call embedding model), agent role (invoke action-group Lambdas, query KBs, invoke its model). Missing role permission is the top cause of "agent/KB suddenly fails" scenarios.
- Lambda action groups also need a resource-based policy allowing the agent to invoke them.
- Guard with SCPs and permission boundaries at org scale; use **IAM Access Analyzer** to find overly broad policies and generate least-privilege policies from CloudTrail activity.

Network isolation — VPC endpoints (PrivateLink)

- Interface endpoints for bedrock-runtime, bedrock-agent-runtime, and the Bedrock control plane keep inference traffic off the public internet; S3 uses a gateway endpoint.
- Endpoint policies further restrict which models/resources an endpoint may reach.
- Data-residency interplay: endpoints keep traffic private per region; cross-region inference profiles (Lesson 02) deliberately route across regions — a compliance tradeoff, not a free upgrade.
- Bedrock itself is a managed service outside your VPC; PrivateLink controls *your* path to it.

KMS — encryption everywhere

- Encrypt S3 source buckets, vector stores, CloudWatch Logs, and invocation-log destinations; customer-managed keys when you need key policies, rotation control, grants, and cross-account access.
- KB/agent service roles need kms:Decrypt/GenerateDataKey on the keys protecting their data — the classic hidden cause of ingestion failures.
- Envelope encryption concept; AWS Encryption SDK for client-side encryption of especially sensitive payloads.

Secrets and app identity

- **Secrets Manager**: Aurora credentials for KB vector stores (Lesson 08), third-party API keys for agent tools; automatic rotation. Never secrets in prompts, code, or environment plaintext.
- **Amazon Cognito**: user pools = sign-in/JWT authorization for your app and API Gateway; identity pools = temporary AWS credentials for direct client access. User authentication; IAM remains service authorization.
- **IAM Identity Center**: workforce SSO for console/CLI access (human operators, not application traffic).

- **WAF**: in front of API Gateway/CloudFront/ALB — rate limiting, managed rule groups, bot control. Stops HTTP abuse; it is **not** a prompt-injection defense (that's Guardrails + validation, Lesson 05).

Finding sensitive data — three tools, three moments

Tool	When	What it does
Amazon Macie	Data **at rest** in S3	Discovers and classifies PII/sensitive data in buckets; findings for remediation before data enters a KB
Amazon Comprehend	Text **in motion**, real-time APIs	PII entity detection/redaction, entities, sentiment, custom classification — pre-filter before the FM
Guardrails sensitive-info filters	**In-flight** at the model boundary	Block/mask PII in prompts and responses (Lesson 05)

Exam pattern: "scan the training/RAG corpus for PII before ingestion" → Macie. "Redact PII from live chat text before it reaches the model" → Comprehend or Guardrail input filter. "Ensure model responses never expose PII" → Guardrail output filter.

Data protection facts about Bedrock itself

- Prompts/responses are **not** used to train Bedrock models; invocation logging is opt-in and goes only to your S3/CloudWatch destinations (Lesson 01).
- Audit: **CloudTrail** records API activity (who created/modified/invoked what); model invocation logs record content when enabled.
- Retention: S3 lifecycle policies expire logs/source data per policy; KB sync propagates source deletions (Lesson 03).
- Tenant isolation pattern: server-side metadata filtering (Lesson 03) + per-tenant IAM-scoped invocation where feasible + tenant-aware cache keys (Lesson 02).

Decision rules

Scenario	Answer
App may invoke only one model	IAM policy scoped to that model/inference-profile ARN
Inference must never traverse the internet	VPC interface endpoints (PrivateLink) + endpoint policies
Ingestion fails after bucket encryption enabled	KB role missing KMS permissions
Find PII across the S3 data lake	Macie
Detect/redact PII in real-time text	Comprehend (pre-processing) or Guardrails (at model boundary)
Store DB/API credentials for agents and KBs	Secrets Manager with rotation
End-user sign-in for the GenAI app	Cognito user pool (+ API Gateway authorizer)

Scenario	Answer
Prove who changed a guardrail/agent and when	CloudTrail

Hands-on checklist

- ☐ IAM policy allowing Converse on exactly one inference profile; verify another model is denied.
- ☐ Create a bedrock-runtime VPC endpoint; invoke a model from a private subnet.
- ☐ Run a Macie scan (or Comprehend PII detect) over sample documents destined for a KB.
- ☐ Reproduce and fix a KMS-permission ingestion failure.

Sources: https://docs.aws.amazon.com/bedrock/latest/userguide/security.html · https://docs.aws.amazon.com/bedrock/latest/userguide/vpc-interface-endpoints.html · https://docs.aws.amazon.com/macie/latest/user/what-is-macie.html

11 — Observability + Cost

Exam focus (Domains 4–5): seeing what your GenAI system did, what it cost, and why quality or latency changed.

What to measure (GenAI-specific)

Traditional metrics (requests, errors, latency) plus FM-specific signals:

- **Token usage**: input/output/cache-read/cache-write tokens per model, per application (inference profile, Lesson 02).
- **Invocation latency** split by stage: retrieval, model, guardrail, downstream tools.
- **Throttling and quota pressure**: ThrottlingException rates, tokens-per-minute headroom.
- **Guardrail interventions**: which policy fired, how often (from guardrail trace data).
- **Quality proxies**: evaluation scores (Lesson 07), hallucination/grounding rates, user feedback (thumbs, ratings), agent task-completion rate.
- **Cost per request / per conversation / per tenant** — the unit economics Domain 4 asks about.

The tool set

CloudWatch

- Bedrock publishes invocation metrics (count, latency, client/server errors, token counts) dimensioned by model and inference profile.
- **Model invocation logging** (opt-in, Lesson 01) sends full request/response payloads to CloudWatch Logs and/or S3 — required for deep debugging, audit, and building evaluation datasets. This is also a cost and privacy decision (Lessons 05/10).
- Logs Insights queries over invocation logs: find the prompts behind latency spikes, sample low-quality responses.
- Alarms + anomaly detection: token-burst anomalies (runaway agent loops show up here first), latency/error thresholds.
- Custom metrics/dashboards: publish app-side quality metrics (grounding pass rate, cache hit rate) via Embedded Metric Format; one dashboard per application.

X-Ray

- End-to-end tracing across API Gateway → Lambda → Bedrock/KB/agent calls; service map shows which hop owns the latency.
- Exam cue: "the chatbot is slow but the model is fast" → trace breaks down retrieval vs invocation vs tool execution.

CloudTrail

- API audit trail: who created/modified/deleted agents, KBs, guardrails, prompts; who invoked what, when. Governance and incident forensics (Lesson 10).
- CloudTrail does **not** contain prompt/response content — that's invocation logging. Confusing the two is a classic distractor.

CloudWatch Synthetics

- Canaries that periodically call your GenAI endpoint with golden prompts and alert on failures or response drift — production smoke tests.

Cost visibility and control

- **Application inference profiles + tags** (Lesson 02) → per-team/app token spend in **Cost Explorer** and the Cost and Usage Report; activate the tags as cost-allocation tags.
- **Cost Anomaly Detection**: alerts on unusual Bedrock spend (e.g., a looping agent burning tokens).
- Optimization loop (from Lesson 02): measure with token metrics → route/cache/prune/batch → re-measure. Cost questions are answered with this loop, not with a single feature.
- Budgets/alerts on forecasted Bedrock spend for guardrails at account level.

Troubleshooting playbooks

Symptom	First places to look
Sudden cost spike	Token metrics by inference profile → invocation logs for the top prompts; suspect agent loops or lost caching
Latency regression	X-Ray trace breakdown (retrieval vs model vs tool); check cross-region routing and cache hit rates
Quality complaints, metrics green	Sample invocation logs; run evaluation job (Lesson 07) — separate retrieval vs generation failure (Lesson 03)
Intermittent throttling	Token-per-minute metrics vs quota; SQS buffering / provisioned throughput / cross-region profile (Lessons 02/09)
Agent doing bizarre things	Agent trace (Lesson 04) + invocation logs; check prompt/guardrail version drift
Need to prove compliance	CloudTrail (actions) + invocation logs (content) + guardrail traces

Decision rules

Scenario	Answer
"Show Bedrock spend per business unit"	Application inference profiles + cost allocation tags + Cost Explorer
"Alert us when token spend behaves abnormally"	Cost Anomaly Detection / CloudWatch anomaly alarms
"Which prompts caused last night's latency spike"	Model invocation logs + Logs Insights

Scenario	Answer
"Where is time spent in each request"	X-Ray tracing
"Who changed the production guardrail"	CloudTrail
"Detect runaway agent token burn early"	CloudWatch token metrics + anomaly alarm
"Continuously verify the live endpoint still answers correctly"	Synthetics canary with golden prompts + scheduled evaluation

Hands-on checklist

- ☐ Enable invocation logging; query it with Logs Insights.
- ☐ Dashboard: invocations, p95 latency, input/output tokens, throttles for one model.
- ☐ Tag an inference profile; find its spend in Cost Explorer.
- ☐ Trace one chat request end-to-end in X-Ray.

Sources: https://docs.aws.amazon.com/bedrock/latest/userguide/monitoring.html · https://docs.aws.amazon.com/bedrock/latest/userguide/model-invocation-logging.html

12 — SageMaker AI Family

Exam focus: when Bedrock's managed path isn't enough — custom models, fine-tuning control, and the ML-platform services that support GenAI governance. Model *training* internals are out of scope; deployment and lifecycle are in scope.

SageMaker vs Bedrock — the master decision

	Bedrock	SageMaker AI
Model access	Managed APIs; no infra	You deploy and run model endpoints
Customization	SFT / distillation / RFT on supported models (Lesson 07)	Full control: full fine-tuning, LoRA/adapters, any open/custom weights
Serving	On-demand / provisioned throughput	Real-time, serverless, async, batch endpoints; GPU choice; autoscaling
Ops burden	Minimal	Endpoint lifecycle, instance sizing, scaling, patching
Choose when	Managed models meet the need	Model not on Bedrock, custom weights, serving control, existing ML platform

Exam default: Bedrock first; SageMaker when the scenario demands control Bedrock doesn't offer.

Components that matter for AIP-C01

JumpStart

One-click deploy and fine-tune of popular foundation models (open and proprietary) to SageMaker endpoints. Fastest SageMaker path to a hosted FM; also the entry point for models not available on Bedrock.

Endpoints (serving options)

- **Real-time endpoints**: always-on, GPU instance choice, autoscaling — custom/fine-tuned model inference.
- **Serverless inference**: sporadic/low traffic, no instance management, cold starts.
- **Asynchronous inference**: long-running requests, queued, S3-based results.
- **Batch transform**: offline scoring over datasets.
- Multi-model / multi-container endpoints: many models behind one endpoint for cost efficiency.

Model Registry

Versioned model packages with approval workflows (Pending → Approved → deployed). The governance answer for self-managed models: which version is in production, rollback to the previous approved version. Pairs with SageMaker Pipelines (CI/CD for model build/deploy) and evaluation gates (Lesson 07).

Clarify

Bias detection and explainability: pre-training data bias, post-training prediction bias, and SHAP-based feature attribution. Exam role: fairness evaluation and bias drift monitoring for governance (Domain 3) — "demonstrate the model treats groups equitably / explain its decisions".

Model Monitor

Baselines from training/reference data; scheduled monitoring of data quality, model quality, bias drift, and feature attribution drift on live endpoints, with CloudWatch alerts. Exam role: "detect model/data drift in production" for SageMaker deployments (for Bedrock, the equivalents are evaluations + invocation-log analysis, Lessons 07/11).

Ground Truth + Amazon Augmented AI (A2I)

- **Ground Truth**: managed data-labeling with human workforces; creates training/evaluation datasets.
- **A2I**: human-in-the-loop review of model predictions at runtime — low-confidence outputs routed to humans; approval/correction workflows. GenAI pattern: human review of sensitive agent actions or generated content (often orchestrated by Step Functions, Lesson 09).

Supporting pieces (recognition level)

- **Data Wrangler**: visual data prep and data-quality analysis for FM-consumption pipelines (Domain 1 data validation).
- **Processing jobs**: Spark/scikit-style preprocessing at scale (e.g., preparing corpora for embedding/fine-tuning).
- **Feature Store, Pipelines, Studio / Unified Studio**: feature management, ML CI/CD, and the IDE/workspace — know they exist and their one-line job (Lesson 13 covers the sweep).
- **Neo**: compiles models for edge/device targets — rarely central; recognition only.

Decision rules

Scenario	Answer
Deploy an open-weight model with our own LoRA adapter	SageMaker (JumpStart/endpoint) + Model Registry
Model unavailable on Bedrock; need GPU serving control	SageMaker real-time endpoint
Track/approve/roll back custom model versions	Model Registry (+ Pipelines)
Detect bias and explain predictions	Clarify
Alert when live model behavior drifts	Model Monitor
Humans must review low-confidence outputs	A2I (often via Step Functions)
Build labeled datasets with a workforce	Ground Truth

Scenario	Answer
Standard RAG/chat with managed models	Bedrock — not SageMaker

Hands-on checklist

- ☐ Deploy a JumpStart model to a real-time endpoint; invoke it; delete it (cost).
- ☐ Register two model versions in Model Registry; approve one; note the deployment/rollback mechanics.
- ☐ (Read) A Clarify bias report and a Model Monitor drift report — know what each metric means.

Source: https://docs.aws.amazon.com/sagemaker/latest/dg/whatis.html

13 — Recognition Sweep

Exam focus: services you only need to recognize in one line — what it is, and the scenario cue that makes it the answer (or reveals it as a distractor). If a service below is ever the *core* of a question, it's usually one of these cues.

Specialized AI services (vs "just use Bedrock")

Service	One line	Cue
Amazon Comprehend	NLP APIs: entities, sentiment, PII detection/redaction, custom classification	Real-time text analytics/redaction without an FM (Lesson 10)
Amazon Textract	OCR + forms/tables extraction from documents	"Extract fields from scanned invoices" (feeds RAG pipelines)
Amazon Transcribe	Speech → text	Call recordings, meeting audio → then summarize with Bedrock
Amazon Rekognition	Image/video analysis and content moderation	"Moderate user-uploaded images"
Amazon Lex	Conversational chatbots (ASR + NLU), pairs with Connect	Structured voice/chat bots with intents and slots
Amazon Kendra	Managed enterprise search with ranking	Traditional enterprise search; KBs are the GenAI successor for most cases
Amazon Q Business	Managed enterprise assistant over company data sources	"Employees ask questions across Confluence/SharePoint/Slack" with minimal build
Amazon Q Developer	AI coding assistant (IDE/CLI)	Developer productivity, code generation/refactoring, AWS guidance
Kiro	Agentic AI IDE (spec-driven development)	Recognize as AWS's AI IDE
Amazon Quick	Agentic business-assistant/research offering	Recognition only
Amazon Connect	Cloud contact center	Voice/chat support with Lex, Transcribe, and Bedrock summaries
Amazon Augmented AI (A2I)	Human review loops for model output	Covered in Lesson 12

Analytics and data

Service	One line	Cue
Amazon Athena	Serverless SQL over S3	Ad-hoc queries over logs/data lake
AWS Glue	ETL + Data Catalog + data-quality checks	Prepare/catalog data; data lineage for governance
Amazon EMR	Managed Spark/Hadoop	Very large-scale data processing
Amazon Kinesis	Real-time data streaming	Streaming ingestion/analytics
Amazon MSK	Managed Kafka	Existing Kafka workloads

Service	One line	Cue
Amazon QuickSight	BI dashboards	Business-facing dashboards over app metrics
Amazon OpenSearch Service	Search/vector platform	Lesson 08
Amazon Managed Grafana	Managed Grafana dashboards	Dashboards over CloudWatch/Prometheus

Compute and containers (where custom GenAI code runs)

Service	One line	Cue
Amazon EC2	VMs, incl. GPU instances	Self-hosted models, full OS control
Amazon ECS / EKS / Fargate	Container orchestration (AWS-native / Kubernetes / serverless containers)	Long-running MCP servers, custom model servers, agent runtimes (Lesson 04)
Amazon ECR	Container image registry	Storing agent/model container images
AWS App Runner	Simplest managed container web service	Small GenAI web apps/APIs without infra
Lambda@Edge / CloudFront Functions	Code at CDN edge	Edge personalization/auth (rarely central)
AWS Outposts	AWS infrastructure on-premises	Data must stay on-prem while using AWS services
AWS Wavelength	Telco-edge compute	Ultra-low-latency edge inference

Databases and storage (beyond Lessons 03/08/09)

Service	One line	Cue
Amazon RDS / Aurora	Relational DBs (Aurora = Lesson 08 pgvector)	Structured app data; structured KBs (Redshift/Glue path, Lesson 03)
Amazon DocumentDB	MongoDB-compatible document DB	Existing MongoDB workloads
Amazon ElastiCache	Redis/Valkey-compatible cache	Session cache, semantic cache backing store, rate limiting (Lesson 02)
DynamoDB Streams	Change stream from DynamoDB	Trigger processing on state changes (Lesson 09)
Amazon EBS / EFS	Block / shared file storage for EC2/ECS	Model artifacts, shared files for compute
S3 Intelligent-Tiering / Lifecycle / Cross-Region Replication	Storage cost, retention, and DR controls	Archive logs, expire data per retention policy, replicate KB sources

Integration, developer tools, and delivery

Service	One line	Cue
Amazon AppFlow	No-code SaaS→AWS data flows (Salesforce etc.)	Ingest SaaS data into S3 for a KB

Service	One line	Cue
AWS AppConfig	Managed configuration/feature flags	Model routing config, kill switches (Lesson 02)
AWS Amplify	Frontend hosting + auth/data for web/mobile	Ship the chat UI fast
AWS AppSync	Managed GraphQL APIs (+ real-time subscriptions)	GraphQL front door for GenAI apps
CloudFormation / AWS CDK	Infrastructure as code	Provision GenAI stacks repeatably; exam prefers IaC answers for scale
CodeBuild / CodePipeline / CodeDeploy (+ CodeArtifact)	CI/CD	Deploy prompts/agents/apps with eval quality gates (Lesson 07)
AWS CLI / SDKs / Tools	Programmatic access	Boto3 etc. — how everything above is automated
AWS X-Ray	Distributed tracing	Lesson 11

Management, governance, and migration

Service	One line	Cue
AWS Auto Scaling	Automatic capacity scaling	Endpoint/app scaling policies
AWS Systems Manager	Ops management; Parameter Store for config/secrets-adjacent values	Fleet/config operations (Secrets Manager still wins for rotatable secrets)
AWS Service Catalog	Curated, approved product templates	"Only approved GenAI stacks may be deployed"
AWS Well-Architected Tool (+ GenAI Lens)	Architecture reviews against best practice	Design-review scenarios (Domain 1)
AWS Chatbot	CloudWatch alarms into Slack/Teams	Alerting channel
AWS DataSync	Managed data movement into AWS	Migrate file corpora into S3 for ingestion
AWS Transfer Family	SFTP/FTP file ingestion	Partners upload documents via SFTP
CloudWatch Synthetics / Cost Explorer / Cost Anomaly Detection	Canaries / spend analysis / spend alerts	Lesson 11

Networking front doors

Service	One line	Cue
Amazon CloudFront	CDN + edge TLS/WAF	Global low-latency delivery of the GenAI app
Elastic Load Balancing (ALB/NLB)	Load balance ECS/EC2 backends	Fronting containerized inference/agent services
AWS Global Accelerator	Anycast entry for global apps	Global performance/availability front door

Service	One line	Cue
Amazon Route 53	DNS + routing policies	Failover/latency-based routing for app endpoints

How to use this file in the final week

- Read the cue column only. Any cue you can't map back to a lesson = revisit that lesson.
- On the exam, these services are usually **distractors** ("sounds plausible, solves the wrong moment"). The correct answer is normally the Lesson 01–12 service whose cue matches the scenario's constraint words.

Source: https://docs.aws.amazon.com/aws-certification/latest/ai-professional-01/aip-01-in-scope-services.html